

Training Guide — From Procedural to Object-Oriented PLC Programming

Based on: *OOP Introduction Parts 1 & 2* by Ben Harrison (Beckhoff Australia / Coding Bytes)

Platform: TwinCAT 3 Structured Text (IEC 61131-3)

Example machine: Conveyor sorting machine

Learning Objectives

By the end of this guide you will be able to:

1. Explain the difference between procedural and object-oriented programming in PLC context
 2. Identify which parts of a procedural program are natural candidates for objects
 3. Create function blocks that act as classes — with properties and methods instead of inputs/outputs
 4. Define and implement interfaces to decouple logic from hardware specifics
 5. Use `FB_init` as a constructor to inject dependencies
 6. Read and reason about a fully OOP-structured TwinCAT 3 project
-

Part 1 — The Paradigm Shift

What is Procedural Programming?

Procedural programming is laying out a sequence of steps. You define inputs, outputs, and the procedure that connects them. In a PLC this is the familiar pattern: global variables for I/O, a long MAIN program that checks conditions and sets outputs, maybe some function blocks for repeated chunks.

There is nothing wrong with procedural programming for simple machines. The problems emerge at scale and over time:

- Long MAIN programs become hard to read quickly
- Conditions like `NOT GVL.StopButton` require knowledge of hardware wiring to interpret
- Duplicating a machine means duplicating all its code and renaming every variable
- Changing a sensor type means finding and editing every reference to that sensor in the logic
- Highly dependent code becomes fragile — touching one thing can break another

This is sometimes called **spaghetti code**: everything tangled together, highly dependent, hard to change without breaking something else.

What is Object-Oriented Programming?

OOP is a method of *encapsulating* code into small, self-contained units called **objects** (or **classes**). Each object:

- Owns its own internal state (variables)
- Exposes a clean interface (methods and properties)

- Hides its implementation details from the outside world
- Does not need to know what is going on outside of itself

When objects do not know too much about each other, they are **decoupled**. Decoupled code can be changed on one side without breaking the other.

The goal is not to write less code — you often write *more* files. The goal is to write code that is:

- **Easy to read** — someone unfamiliar with the machine can reason about it
- **Easy to change** — modifying one object does not cascade through everything else
- **Reusable** — a well-made class can be dropped into any project that needs it

Why OOP Feels Like a Natural Progression

Procedural PLC programmers already use function blocks to avoid repeating logic. OOP is the next step: instead of passing all inputs and outputs through the function block's VAR_INPUT/VAR_OUTPUT, you treat the function block as a **class** — a self-contained object that manages its own state and talks to the world through methods and properties.

"OOP is almost a natural progression of procedural programming. I find it very hard to go back — it's almost like a method you can use to simplify your code and make things very reusable, but fits quite nicely with procedural programming."

— Ben Harrison

Function Blocks as Classes

In IEC 61131-3 Structured Text, function blocks are used as classes. The terminology maps directly:

OOP concept	TwinCAT 3 equivalent
Class	FUNCTION_BLOCK
Object / instance	Instantiated function block variable
Constructor	FB_init method
Method	METHOD
Property	PROPERTY with GET / SET accessors
Interface	INTERFACE (.TcIO file)
Implements	IMPLEMENTS keyword on the function block

One important difference: a standard PLC function block is called cyclically with inputs and outputs. A class-style function block removes those VAR_INPUT / VAR_OUTPUT sections. External code never calls the body directly — it only interacts through methods and properties.

Hungarian Notation

Part 2 — The Starting Point

The diagram illustrates a conveyor sorting machine with the following components and labels:

- Conveyor Sorting Machine**: The main title of the system.
- Control panel**: A black panel with a yellow border containing three buttons: a green **Start** button, a red **Stop** button, and a red **Emergency Stop** button.
- Product Detected Sensor**: A yellow sensor unit mounted on a vertical post at the start of the conveyor.
- Conveyor Full Sensor**: Two yellow sensor units, one mounted on the side of the conveyor belt and another on the end of the conveyor.
- Motor Contactor**: Two black electrical components, one located near the end of the conveyor and another near the motor.

[illegible]

1. Press Start while Stop is released → enter run mode
2. Press Stop or E-Stop → leave run mode immediately

3. While in run mode: if product is detected, run the motor for at least 5 seconds after the last product (off-delay timer)
4. If the destination conveyor is full, inhibit the motor even in run mode

The Procedural Version (`PLC_NoOOP`)

GVL:

```
{attribute 'qualified_only'}
VAR_GLOBAL
    EmergencyStopButton    AT%I* : BOOL;
    StopButton             AT%I* : BOOL;
    StartButton            AT%I* : BOOL;
    ProductDetectedSensor  AT%I* : BOOL;
    ConveyorFullSensor     AT%I* : BOOL;
    MotorContactor        AT%Q* : BOOL;
END_VAR
```

MAIN:

```
PROGRAM MAIN
VAR
    ConveyorControl : BOOL;
    Timeout         : TOF;
END_VAR

IF NOT GVL.StopButton OR NOT GVL.EmergencyStopButton THEN
    ConveyorControl := FALSE;
ELSIF gvl.StartButton AND gvl.StopButton THEN
    ConveyorControl := TRUE;
END_IF

Timeout(IN := ConveyorControl AND gvl.ProductDetectedSensor, PT := T#5S, Q =>, ET =>);

gvl.MotorContactor := ConveyorControl AND Timeout.Q AND NOT gvl.ConveyorFullSensor;
```

This works and is easy to understand once you know the hardware. But notice:

- `NOT GVL.StopButton` — the NOT is a hardware detail, not a logic intent. Someone reading this code must know the button is normally closed to understand why we invert it.
- `gvl.StartButton AND gvl.StopButton` — the stop button appears in the start condition (because it must be released = TRUE for a NC button). This is confusing without context.
- `NOT gvl.ConveyorFullSensor` — same hardware-knowledge dependency.
- Every variable is a flat BOOL. The GVL is a list of terminals, not a description of the machine.
- Adding a second conveyor requires copying and renaming everything.

Part 3 — Step-by-Step Refactoring

This section follows the exact progression from the tutorial videos. Each step introduces one OOP concept, shows the code change, and explains what you gain.

Step 1 — Identify Your Objects

Before writing any OOP code, look at the machine and identify the real-world things that could become objects. Start with the physical devices — the things you can touch.

In this machine:

- Start button (normally open push button)
- Stop button (normally closed push button)
- Emergency stop button (normally closed push button)
- Product detected sensor (normally closed)
- Conveyor full sensor (normally closed)
- Motor contactor

These are all good candidates because:

- They have a clear identity and purpose
- They own internal state (their I/O address)
- They have a behaviour that varies by type (normally open vs normally closed)

Create a **Devices** folder in your PLC project. This is where device-level classes will live.

Step 2 — Create **NormallyOpenPushButton**

Add a new function block in **Devices/NormallyOpenPushButton.TcPOU**. Remove the **VAR_INPUT** and **VAR_OUTPUT** sections — this is a class, not a traditional function block.

```
{attribute 'no_explicit_call' := 'NormallyOpenPushButton is a CLASS and must be accessed using
methods or properties'}
FUNCTION_BLOCK NormallyOpenPushButton
VAR
    input AT %I* : BOOL;
END_VAR
```

The hardware input address now lives *inside* the class. When you instantiate this function block anywhere in your project, it automatically comes with its own input variable. You never forget to map the hardware.

Step 3 — Add Domain Language with Properties

Rather than evaluating **startButton = TRUE**, we want to say **startButton.IsPressed**. This is **domain language** — terms that make sense to the machine's domain, not just to a software engineer.

Right-click the function block → Add → Property → **IsPressed**, return type **BOOL**, public, getter only (no setter — a button's pressed state is read-only from outside).

```
PROPERTY IsPressed : BOOL
// GET body:
IsPressed := input;
```

For a normally open button, the input is TRUE when pressed. So `IsPressed := input` is correct.

Add a second property `IsReleased`:

```
PROPERTY IsReleased : BOOL
// GET body:
IsReleased := NOT input;
```

Notice that `IsReleased` refers to the input directly. You could also write `IsReleased := NOT IsPressed` — using one property to define another. The object owns all this knowledge internally.

Step 4 — Create `NormallyClosedPushButton`

Copy `NormallyOpenPushButton` and create `NormallyClosedPushButton`. Only the property getters change:

```
// NormallyClosedPushButton
PROPERTY IsPressed : BOOL → IsPressed := NOT input; // circuit opens when pressed
PROPERTY IsReleased : BOOL → IsReleased := input; // circuit closed = released
```

The difference between normally open and normally closed is now entirely *inside* the class. Code that uses these objects through `IsPressed` / `IsReleased` never needs to know which type it has.

Step 5 — Update the GVL

Replace the raw BOOL variables with the new class types:

```
VAR_GLOBAL
    emergencyStopButton : NormallyClosedPushButton; // was: AT%I* BOOL
    stopButton          : NormallyClosedPushButton; // was: AT%I* BOOL
    startButton         : NormallyOpenPushButton;   // was: AT%I* BOOL
    ...
END_VAR
```

The GVL is starting to look like a **parts list**. You can read it and immediately know what hardware is on the machine. The hardware addresses are still there but hidden inside each object.

Step 6 — Update MAIN to Use Properties

Replace the raw boolean conditions with property calls:

Before:

```

IF NOT GVL.StopButton OR NOT GVL.EmergencyStopButton THEN
    ConveyorControl := FALSE;
ELSIF gvl.StartButton AND gvl.StopButton THEN
    ConveyorControl := TRUE;
END_IF

```

After:

```

IF GVL.stopButton.IsPressed OR GVL.emergencyStopButton.IsPressed THEN
    ConveyorControl := FALSE;
ELSIF gvl.startButton.IsPressed AND gvl.stopButton.IsReleased THEN
    ConveyorControl := TRUE;
END_IF

```

Read both versions aloud. The second version reads like a specification. The NOT operators are gone — the normally-closed logic is encapsulated inside each object. `stopButton.IsReleased` communicates intent far more clearly than `gvl.StopButton` (which is TRUE when not pressed because it's normally closed).

What you gain: Change a button from normally closed to normally open? Change one line in the GVL. MAIN does not change.

Step 7 — Create Sensor Classes

Follow the same pattern for sensors. Create `NormallyOpenSensor` and `NormallyClosedSensor` in the `Devices` folder, each with `IsActive` and `IsNotActive` properties.

```

// NormallyClosedSensor
PROPERTY IsActive      : BOOL → IsActive      := input;
PROPERTY IsNotActive   : BOOL → IsNotActive   := NOT input;

```

Update the GVL:

```

productDetectedSensor : NormallyClosedSensor;
conveyorFullSensor    : NormallyClosedSensor;

```

Update MAIN:

```

// Before:
Timeout(IN := ConveyorControl AND gvl.ProductDetectedSensor, ...);
gvl.MotorContactor := ConveyorControl AND Timeout.Q AND NOT gvl.ConveyorFullSensor;

// After:
Timeout(IN := ConveyorControl AND gvl.productDetectedSensor.IsActive, ...);
IF ConveyorControl AND Timeout.Q AND gvl.conveyorFullSensor.IsNotActive THEN

```

The `NOT` on `ConveyorFullSensor` is now gone. `IsNotActive` expresses the intent: "run the motor if the conveyor is not full."

What you gain: Swapping `conveyorFullSensor` from `NormallyClosedSensor` to `NormallyOpenSensor` in the GVL automatically reverses the polarity. No change to MAIN required.

Step 8 — Create `Contactor` (Methods)

Create an `Actuators` folder. Add `Contactor` as a function block with methods instead of a raw output assignment.

```
{attribute 'no_explicit_call' := 'Contactor is a CLASS and must be accessed using methods or properties'}
FUNCTION_BLOCK Contactor
VAR
    output AT %Q* : BOOL;
END_VAR

METHOD SwitchOn → output := TRUE;
METHOD SwitchOff → output := FALSE;
```

The output address lives inside the contactor. The hardware mapping for the entire motor circuit is now a single entry: `GVL.conveyorMotor.contactor.output`. This self-documents where the coil is wired without any comments.

Properties vs Methods: Properties are for state you can read or write (e.g. `IsPressed`, `IsActive`). Methods are for actions you ask an object to perform (e.g. `SwitchOn()`, `Start()`). Methods can take parameters and return values; in this case neither is needed.

Step 9 — Create `DirectOnlineStarterMotor`

Add `DirectOnlineStarterMotor` to the `Actuators` folder. It *composes* a `Contactor` and adds a layer of motor-level language:

```
{attribute 'no_explicit_call' := 'DirectOnlineStarterMotor is a CLASS and must be accessed using methods or properties'}
FUNCTION_BLOCK DirectOnlineStarterMotor
VAR
    contactor : Contactor;
END_VAR

METHOD Start → contactor.SwitchOn();
METHOD Stop → contactor.SwitchOff();
```

Update the GVL:

```
conveyorMotor : DirectOnlineStarterMotor;
```

Update MAIN's motor control:


```
// Before:
gvl.MotorContactor := ConveyorControl AND Timeout.Q AND NOT gvl.ConveyorFullSensor;

// After (with IF/ELSE because methods cannot be used in assignment):
IF ConveyorControl AND Timeout.Q AND gvl.conveyorFullSensor.IsNotActive THEN
    gvl.conveyorMotor.Start();
ELSE
    gvl.conveyorMotor.Stop();
END_IF
```

MAIN now talks to the motor as a motor, not as a contactor coil. Whether the motor is DOL, inverter-driven, or servo is hidden inside the object.

Composition: `DirectOnlineStarterMotor` owns a `Contactor`. When you instantiate a `DirectOnlineStarterMotor`, you automatically get a contactor with it. The hardware output address is allocated automatically as `motor.contactor.output` — TwinCAT builds the I/O tree from the nested object structure.

Part 4 — Introducing Interfaces

Steps 1–9 gave us encapsulation and domain language. Interfaces add the next dimension: **the ability to swap concrete types at any scope level without changing the code that uses them.**

What is an Interface?

An interface is a **contract**. It declares a set of properties and methods that any implementing class *must* provide. The interface itself contains no code — only signatures.

```
INTERFACE I_PushButton
    PROPERTY IsPressed : BOOL (GET only)
    PROPERTY IsReleased : BOOL (GET only)
```

Any function block that declares `IMPLEMENTS I_PushButton` must provide both `IsPressed` and `IsReleased` properties. If it does not, the compiler refuses to build — the contract is enforced at design time.

An interface variable (e.g. `stopButton : I_PushButton`) is a reference — it points to an object that implements the interface. The type of that object does not matter as long as it obeys the contract.

Step 10 — Create `I_PushButton` Interface

Add a new interface in `Devices/I_PushButton.TcIO`:

```
INTERFACE I_PushButton
    PROPERTY IsPressed : BOOL (GET only)
    PROPERTY IsReleased : BOOL (GET only)
```

Then declare that both button classes implement it:

```
FUNCTION_BLOCK NormallyOpenPushButton IMPLEMENTS I_PushButton
FUNCTION_BLOCK NormallyClosedPushButton IMPLEMENTS I_PushButton
```

Build the project. TwinCAT verifies that both classes satisfy the contract. If you add a property to the interface and forget to implement it in a class, the build fails with a clear error.

Step 11 — Create `I_Sensor` and `I_SingleDirectionMotor`

Follow the same pattern:

```
INTERFACE I_Sensor
    PROPERTY IsActive      : BOOL  (GET only)
    PROPERTY IsNotActive   : BOOL  (GET only)

INTERFACE I_SingleDirectionMotor
    METHOD Start()
    METHOD Stop()
```

Apply to the classes:

```
FUNCTION_BLOCK NormallyClosedSensor      IMPLEMENTS I_Sensor
FUNCTION_BLOCK NormallyOpenSensor        IMPLEMENTS I_Sensor
FUNCTION_BLOCK DirectOnLineStarterMotor  IMPLEMENTS I_SingleDirectionMotor
```

Step 12 — Create `I_Module` Interface

A "module" is a self-contained unit of machine functionality that can be turned on and off. The machine controller does not need to know how the module works internally — it just needs to enable or disable it.

```
INTERFACE I_Module
    METHOD Enable()
    METHOD Disable()
```

This interface concept is powerful because it allows the machine controller to manage completely different kinds of modules (a sorting conveyor, a weighing station, a labelling unit) through a single, uniform API.

Step 13 — Create `SortingConveyor` Module

Create a `Modules` folder. Add `SortingConveyor` implementing `I_Module`. This class moves all the conveyor timing logic out of MAIN and into its own encapsulated object.

Key insight — dependency injection: Instead of the `SortingConveyor` owning specific sensor and motor types, it receives *interfaces*. It does not know or care whether the sensor is normally open or normally closed, or whether the motor is DOL or inverter-driven.

```

FUNCTION_BLOCK SortingConveyor IMPLEMENTS I_Module
VAR
    productDetected : I_Sensor;
    conveyorFull    : I_Sensor;
    conveyor        : I_SingleDirectionMotor;
    enabled          : BOOL;
    timeout          : TOF;
END_VAR

// Body – the same logic that was in MAIN
Timeout(IN := enabled AND productDetected.IsActive, PT := T#5S, Q =>, ET =>);
IF enabled AND Timeout.Q AND conveyorFull.IsNotActive THEN
    conveyor.Start();
ELSE
    conveyor.Stop();
END_IF

METHOD Enable → enabled := TRUE;
METHOD Disable → enabled := FALSE;

```

The body is under 10 lines. This is an important target in OOP. Short code blocks are easy to read and very hard to write bugs into.

The `FB_init` Constructor

The interface variables (`productDetected`, `conveyorFull`, `conveyor`) start empty — they point to nothing. You must inject concrete objects into them before the code runs. This is done in `FB_init`:

```

METHOD FB_init : BOOL
VAR_INPUT
    bInitRetains    : BOOL;
    bInCopyCode     : BOOL;
    productDetected : I_Sensor;
    conveyorFull    : I_Sensor;
    conveyor        : I_SingleDirectionMotor;
END_VAR
    THIS^.productDetected := productDetected;
    THIS^.conveyorFull    := conveyorFull;
    THIS^.conveyor        := conveyor;

```

Why `THIS^`?

The parameter names in `FB_init` are the same as the member variable names. Without `THIS^`, the compiler sees `productDetected := productDetected` and resolves both to the local parameter — a no-op. `THIS^` is a pointer to the current object instance. `THIS^.productDetected` explicitly refers to the *member variable*, not the parameter.

If you name the parameters differently (e.g. `newProductDetected`) you do not need `THIS^`. Many developers prefer this style to avoid confusion. The tutorial uses same-name parameters to demonstrate that `THIS^` exists and is necessary in this case.

Step 14 — Create `ConveyorSortingMachine`

Create a `Machines` folder. `ConveyorSortingMachine` is the top-level controller. It receives interface references for all three buttons and the conveyor module, and translates button events into module commands.

```
FUNCTION_BLOCK ConveyorSortingMachine
VAR
    emergencyStopButton : I_PushButton;
    stopButton          : I_PushButton;
    startButton         : I_PushButton;
    conveyorModule      : I_Module;
END_VAR

IF StopButton.IsPressed OR EmergencyStopButton.IsPressed THEN
    conveyorModule.Disable();
ELSIF StartButton.IsPressed AND StopButton.IsReleased THEN
    conveyorModule.Enable();
END_IF
```

This is exactly the same logic as PLC_NoOOP's MAIN start/stop section — but now it reads at the domain level. No `NOT` operators. No hardware wiring knowledge required. Someone reading this for the first time immediately understands what it does.

The constructor follows the same `FB_init` pattern:

```
METHOD FB_init : BOOL
VAR_INPUT
    bInitRetains      : BOOL;
    bInCopyCode       : BOOL;
    emergencyStopButton : I_PushButton;
    stopButton        : I_PushButton;
    startButton       : I_PushButton;
    conveyorModule     : I_Module;
END_VAR

THIS^.emergencyStopButton := emergencyStopButton;
THIS^.stopButton          := stopButton;
THIS^.startButton         := startButton;
THIS^.conveyorModule      := conveyorModule;
```

Step 15 — Wire Everything in the GVL

The GVL is now the **composition root** — the one place where concrete objects are created and wired together. Declaration order matters: a variable must exist before it can be passed as an argument.

```

{attribute 'qualified_only'}
VAR_GLOBAL
    // Concrete devices (declaration order: lower layers first)
    emergencyStopButton : NormallyClosedPushButton;
    stopButton          : NormallyClosedPushButton;
    startButton         : NormallyOpenPushButton;
    productDetectedSensor : NormallyClosedSensor;
    conveyorFullSensor   : NormallyClosedSensor;
    conveyorMotor        : DirectOnlineStarterMotor;

    // Module - receives device references via FB_init
    sortingConveyor : SortingConveyor(
        productDetected := productDetectedSensor,
        conveyorFull    := conveyorFullSensor,
        conveyor        := conveyorMotor);

    // Machine - receives module and device references via FB_init
    myConveyorSortingMachine : ConveyorSortingMachine(
        emergencyStopButton := emergencyStopButton,
        stopButton          := stopButton,
        startButton         := startButton,
        conveyorModule      := sortingConveyor);
END_VAR

```

The parenthesised arguments after the type name are passed to `FB_init` at instantiation time. This is TwinCAT's inline constructor syntax.

Notice that the *concrete* types (`NormallyClosedPushButton`, `NormallyClosedSensor`, `DirectOnlineStarterMotor`) only appear in the GVL. `SortingConveyor` and `ConveyorSortingMachine` only ever see interfaces.

Step 16 — Simplify MAIN

With all logic encapsulated in objects, MAIN becomes two lines:

```

GVL.sortingConveyor();
GVL.myConveyorSortingMachine();

```

These calls give each object a PLC scan cycle to execute its body code. `sortingConveyor()` runs first so the timer and motor state are updated before `myConveyorSortingMachine()` reads the module's enabled state.

There is an ongoing discussion in the OOP PLC community about whether objects should expose an `Update()` method rather than being called directly. The tutorial uses the direct call style for simplicity. Either approach is valid; the important point is that MAIN should not contain any control logic.

Part 5 — Unlocking the Full Power of OOP

The steps above give you cleaner, more readable code. But the real payoff from OOP — especially from interfaces and dependency injection — comes from what you can do *because* of the decoupling.

Swapping Types at Design Time

Need to change all sensors from normally closed to normally open? One line changes in the GVL. No other code changes.

```
// Before:
productDetectedSensor : NormallyClosedSensor;

// After:
productDetectedSensor : NormallyOpenSensor;
```

`SortingConveyor` only knows `I_Sensor`. It does not care which type arrives.

Swapping Objects at Runtime

You can add a method to inject a new sensor reference at runtime. This allows runtime failover to a backup sensor without stopping the machine:

```
METHOD ChangeConveyorFullSensor
VAR_INPUT
    newSensor : I_Sensor;
END_VAR
THIS^.conveyorFull := newSensor;
```

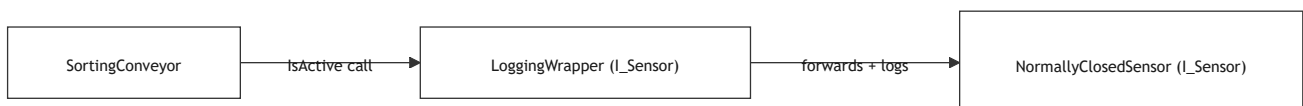
In MAIN or another object, you can now call:

```
IF sensorFailed THEN
    GVL.sortingConveyor.ChangeConveyorFullSensor(GVL.backupConveyorFullSensor);
END_IF
```

The `SortingConveyor` seamlessly switches to the backup sensor at runtime. No other code knows or cares.

Logging and Monitoring Wrappers

Because `SortingConveyor` holds its sensors as `I_Sensor` references, you can insert a wrapper object between the sensor and the conveyor. The wrapper implements `I_Sensor` (so it looks like a sensor to the conveyor) and *contains* an `I_Sensor` (the real sensor it forwards to). In the middle, it logs every `IsActive` call.



`SortingConveyor` does not know a wrapper was inserted. The wrapper does not know it is inside a conveyor. This composability is only possible because of interface-based dependencies.

Reusability and Libraries

Every class in the OOP project (`Contactactor`, `DirectOnlineStarterMotor`, `NormallyClosedSensor`, `SortingConveyor`, etc.) can be placed in a TwinCAT library. Write once, use in every project.

Because `SortingConveyor` does not name any concrete types, it works with any sensors and motor that implement the right interfaces. You could use it with:

- A different motor driver (inverter, servo) that implements `I_SingleDirectionMotor`
- Sensors from any vendor as long as you wrap them in an `I_Sensor` adapter
- Simulated sensors during testing

The effort to write the first OOP project is higher than a procedural equivalent. The second project reuses the library and is significantly faster to build.

Test-Driven Development

Because every dependency is an interface, you can substitute **simulated objects** during testing. Instead of needing physical hardware, create a `SimulatedSensor` and a `SimulatedMotor` that implement `I_Sensor` and `I_SingleDirectionMotor`. You control their state directly from the test.

```
// Test scenario: product detected, conveyor not full → motor should run
sortingConveyorUnderTest : SortingConveyor(
    productDetected := simulatedProductSensor,    // SimulatedSensor: IsActive = TRUE
    conveyorFull     := simulatedFullSensor,      // SimulatedSensor: IsActive = FALSE
    conveyor         := simulatedMotor);

// Enable the conveyor and wait one scan
sortingConveyorUnderTest.Enable();
sortingConveyorUnderTest(); // one scan cycle

// Assert: motor should be running
ASSERT(simulatedMotor.IsRunning = TRUE);
```

This is extremely difficult to do with tightly-coupled procedural code where I/O addresses are hardcoded into the logic.

Debugging in Layered Code

New developers sometimes worry that deeply nested OOP code is harder to debug. In practice, the layering helps.

Scenario: the motor contactor is not energising.

1. Open `GVL.conveyorMotor` — inspect the `Contactor` output. Is it FALSE?
2. If yes, open `GVL.sortingConveyor` — is it calling `Start()`? Is `enabled = TRUE`? Is `Timeout.Q = TRUE`? Is `conveyorFull.IsNotActive = TRUE`?
3. If the conveyor is disabled, open `GVL.myConveyorSortingMachine` — are the buttons in the right state?

Each object is 10 lines or less. You are never reasoning about more than 10 lines at a time. The debugging path is short and deterministic.

"There was a bug — the contactor didn't turn on. You start from the contactor: switched on or off — not been switched on. Who uses the contactor? The direct online starter motor. Who uses that? The sorting conveyor. Seven lines worth of debugging."

— Ben Harrison

Part 6 — The SOLID Connection

The transformation you have just worked through is a practical application of the **D in SOLID: Dependency Inversion Principle**.

The principle states:

- High-level modules should not depend on low-level modules. Both should depend on abstractions.
- Abstractions should not depend on details. Details should depend on abstractions.

In our code:

- `ConveyorSortingMachine` (high-level) depends on `I_PushButton` and `I_Module` (abstractions), not on `NormallyClosedPushButton` or `SortingConveyor` (details).
- `SortingConveyor` depends on `I_Sensor` and `I_SingleDirectionMotor` (abstractions), not on `NormallyClosedSensor` or `DirectOnlineStarterMotor` (details).

The details depend on the abstractions (by implementing the interfaces), not the other way around. This inversion is what gives the code its flexibility.

The other SOLID principles are also visible:

- **S (Single Responsibility):** `Contactor` only manages a coil. `SortingConveyor` only manages sorting conveyor logic.
- **O (Open/Closed):** Add a new sensor type by creating a new class implementing `I_Sensor`. Nothing existing changes.
- **L (Liskov Substitution):** Any `I_Sensor` can replace any other `I_Sensor` wherever one is expected. Used throughout via interfaces.
- **I (Interface Segregation):** `I_Module` only has `Enable` / `Disable`. It does not include sensor reading or motor control — those belong to the implementing class.

Summary

Aspect	Procedural (<code>PLC_NoOOP</code>)	OOP (<code>PLC</code>)
GVL	Flat list of I/O addresses	Parts list of named objects
MAIN	All control logic	Two lines
Sensor type change	Edit logic in MAIN	Change one line in GVL
Second conveyor	Copy and rename everything	Instantiate <code>SortingConveyor</code> with different dependencies
Hardware wiring knowledge in logic	Yes (<code>NOT stopButton</code>)	No (<code>stopButton.IsPressed</code>)

Testability	Hard — I/O addresses are hardcoded	Easy — inject simulated objects
Reusability	Function block copy/paste	Library with interface contracts
Files	2 (GVL + MAIN)	14 (but each is <10 lines)

The OOP version has more files. Every individual file is small, self-contained, and readable in isolation. The complexity has not gone away — it has been organised.

Next Steps

Once comfortable with this example, the natural next topics are:

- **State machines in OOP** — how to implement STATE_MACHINE patterns inside an object without blowing the 10-line budget
- **Libraries** — packaging your device and actuator classes for reuse across projects
- **Extended Interfaces** — having one class implement multiple interfaces (e.g. `I_Sensor` and `I_Loggable`)
- **Runtime polymorphism** — arrays of `I_Module` references that allow the machine to manage a variable number of modules
- **Unit testing frameworks** — TcUnit for automated testing of individual function blocks

For further OOP PLC learning, Ben Harrison recommends the YouTube channel of [Christopher Ohkravi](#) — a software engineer whose videos cover OOP patterns in depth and translate well to PLC programming.